FADENA

LICENCIATURA EN CIBERDEFENSA

DISPOSITIVOS REMOTOS E INTERNET DE LAS COSAS
CÁTEDRA: DIEGO RACERO
AÑO: 2026

CARGA HORARIA: 124 HS

SEGURIDAD EN LAS COMUNICACIONES IoT

Protección esencial para redes y dispositivos conectados

Importancia de la Seguridad en IoT

Contenido:

La seguridad en Internet de las Cosas (IoT) representa uno de los desafíos más críticos de la era digital actual. La superficie de ataque de IoT se expande exponencialmente. Cada dispositivo conectado, desde sensores industriales hasta dispositivos médicos implantables, representa un punto potencial de vulnerabilidad.

La importancia de la seguridad IoT trasciende la protección individual de dispositivos. Un solo dispositivo comprometido puede servir como puerta de entrada para atacar redes enteras, afectando infraestructuras críticas como sistemas de energía, transporte, salud y manufactura. El incidente de Mirai en 2016, donde millones de dispositivos IoT fueron secuestrados para crear una botnet masiva, demostró cómo dispositivos aparentemente inofensivos pueden convertirse en armas cibernéticas.

Además, los dispositivos IoT manejan datos sensibles: información de salud personal, patrones de comportamiento doméstico, datos biométricos y secretos industriales. La falta de seguridad adecuada no solo expone estos datos a robo o manipulación, sino que también puede resultar en consecuencias físicas directas cuando se trata de dispositivos que controlan sistemas críticos.

Riesgos comunes en IoT

Contenido:

Interceptación de datos: Los dispositivos IoT frecuentemente transmiten información a través de redes inalámbricas sin cifrado adecuado. Atacantes pueden emplear técnicas de sniffing para capturar datos en tránsito, desde credenciales de acceso hasta información sensible de usuarios. Protocolos como MQTT o CoAP, aunque eficientes, pueden ser vulnerables si no se implementan correctamente con capas de seguridad adicionales.

Suplantación de identidad (Spoofing): La falta de mecanismos robustos de autenticación permite que atacantes se hagan pasar por dispositivos legítimos o usuarios autorizados. Esto incluye ataques de man-in-the-middle donde el atacante intercepta y potencialmente modifica comunicaciones entre dispositivos, o spoofing de direcciones MAC/IP para infiltrarse en redes.

Acceso físico no autorizado: Muchos dispositivos IoT se despliegan en ubicaciones remotas o de difícil supervisión. Esto los hace vulnerables a manipulación física: extracción de firmware, conexión de interfaces de depuración (JTAG, UART), o incluso reemplazo completo del dispositivo. La falta de protección contra el tamper físico facilita estos ataques.

Vulnerabilidades adicionales:

- Credenciales predeterminadas no modificables
- Actualizaciones de firmware inexistentes o inseguras
- Interfaces web y APIs mal protegidas
- Falta de validación de entrada que permite inyecciones
- Gestión inadecuada de sesiones
- Servicios de red innecesarios expuestos
- Almacenamiento inseguro de datos sensibles en el dispositivo

Soluciones criptográficas y arquitectónicas

Criptografía en IoT:

Confidencialidad: Implementación de cifrado simétrico (AES-128/256) para datos en tránsito y reposo. Para dispositivos con recursos limitados, algoritmos ligeros como ChaCha20 o PRESENT ofrecen balance entre seguridad y eficiencia. El uso de TLS/DTLS para comunicaciones garantiza canales seguros.

Integridad: Códigos de autenticación de mensajes (MAC) y funciones hash criptográficas (SHA-256, SHA-3) aseguran que los datos no han sido alterados. Las firmas digitales basadas en criptografía de curva elíptica (ECC) proporcionan verificación de integridad con menor overhead computacional que RSA.

Autenticación: Certificados digitales X.509 e infraestructuras de clave pública (PKI) permite autenticación mutua entre dispositivos y servicios. Protocolos como OAuth 2.0 y JWT facilitan la autorización granular. Para ambientes constrained, EDHOC y OSCORE ofrecen alternativas eficientes.

Constrained

- Uso en IoT: Los dispositivos IoT a menudo son "constrained", lo que significa que tienen limitaciones en cuanto a energía, procesamiento, memoria y conectividad. Esto implica que los protocolos y tecnologías utilizados en IoT deben ser eficientes y diseñados para funcionar en estas condiciones restrictivas.

EDHOC (Ephemeral Diffie-Hellman Over COSE)

- Uso en IoT: EDHOC es especialmente relevante para IoT porque permite el intercambio seguro de claves entre dispositivos con recursos limitados. Proporciona una forma de establecer una clave compartida sin necesidad de una infraestructura de clave pública compleja, lo que es ideal para dispositivos que pueden no tener la capacidad de manejar operaciones criptográficas complejas.

OSCORE (Object Security for Constrained RESTful Environments)

- Uso en IoT: OSCORE se utiliza para asegurar las comunicaciones en entornos RESTful, que son comunes en IoT. Proporciona mecanismos para proteger la integridad y confidencialidad de los mensajes, lo que es fundamental para garantizar la seguridad de los datos transmitidos entre dispositivos IoT.

Arquitecturas de Seguridad:

Secure Boot y Cadena de Confianza: Verificación criptográfica del firmware durante el arranque, asegurando que solo código autorizado se ejecute. Cada etapa valida la siguiente, estableciendo una cadena de confianza desde el hardware hasta las aplicaciones.

Hardware Security Modules (HSM) y Trusted Execution Environments (TEE): Elementos seguros que almacenan claves criptográficas y ejecutan operaciones sensibles aisladas del sistema operativo principal. Tecnologías como ARM TrustZone o secure elements discretos (ATECC608, etc.).

Arquitectura Zero Trust: Segmentación de red, microsegmentación, y verificación continua de identidad y contexto. Ningún dispositivo es confiable por defecto, independientemente de su ubicación en la red.

Device Identity y Lifecycle Management: Gestión de identidades únicas por dispositivo desde fabricación hasta desmantelamiento. Incluye aprovisionamiento seguro, rotación de credenciales, y revocación cuando es necesario.

Over-The-Air (OTA) Updates seguros: Mecanismos para actualización remota de firmware con verificación de autenticidad, integridad, y capacidad de rollback en caso de fallos.

Seguridad como proceso continuo

La seguridad en IoT no puede conceptualizarse como un estado final alcanzable, sino como un proceso dinámico que requiere adaptación constante. Este enfoque se fundamenta en varios principios:

Evolución de amenazas: Los vectores de ataque se sofistican continuamente. Vulnerabilidades zero-day emergen regularmente, y técnicas que antes eran teóricas se vuelven prácticas con el avance tecnológico (como la computación cuántica amenazando la criptografía actual). Los sistemas IoT deben diseñarse anticipando que serán atacados de formas aún desconocidas.

Ciclo de vida del dispositivo: Un dispositivo IoT puede operar durante 5-6 años en campo. Durante este período, debe mantener actualizaciones de seguridad, gestión de

vulnerabilidades descubiertas post-despliegue, y adaptación a nuevos estándares de seguridad. Esto requiere planificación desde el diseño inicial.

Monitoreo y respuesta a incidentes: Implementación de sistemas SIEM (Security Information and Event Management) adaptados a IoT, detección de anomalías mediante machine learning, y protocolos claros de respuesta ante brechas de seguridad. La telemetría de seguridad debe ser recolectada y analizada continuamente.

Security by Design: Integración de consideraciones de seguridad desde las etapas iniciales de diseño, no como agregado posterior. Esto incluye modelado de amenazas, análisis de superficie de ataque, y testing de penetración regular.

Frameworks y estándares evolutivos: Adopción de marcos como IEC 62443 (seguridad industrial), ISO/IEC 27001, o ETSI EN 303 645 que se actualizan para reflejar el paisaje de amenazas actual. Participación en comunidades de seguridad y programas de divulgación responsable de vulnerabilidades.

Cultura organizacional: La seguridad IoT requiere compromiso desde la alta dirección hasta desarrolladores y operadores. Capacitación continua, awareness de seguridad, y recursos dedicados son fundamentales para mantener postura de seguridad efectiva.

Protección de la comunicación con TLS

Transport Layer Security (TLS) es el protocolo criptográfico fundamental para asegurar comunicaciones en redes IoT. Actúa como una capa de seguridad entre la capa de aplicación y la capa de transporte, creando un "túnel" cifrado que protege los datos en tránsito.

Establecimiento del canal cifrado:

El proceso de handshake de TLS establece una sesión segura mediante los siguientes pasos:

1. **Negociación de versión y cipher suites:** El cliente y servidor acuerdan qué versión de TLS usar y qué algoritmos criptográficos emplear (cifrado simétrico, hash, intercambio de claves).
2. **Autenticación del servidor:** El servidor presenta su certificado digital X.509, que contiene su clave pública y está firmado por una Autoridad Certificadora (CA) confiable. El cliente verifica la validez del certificado, incluyendo:
    - Firma de la CA
    - Período de validez
    - Identidad del servidor (Common Name o Subject Alternative Name)
    - Estado de revocación (mediante OCSP o CRL)
3. **Autenticación mutua (opcional pero recomendada en IoT):** En escenarios IoT, es crítico que el servidor también autentique al dispositivo cliente mediante certificados, previniendo conexiones de dispositivos no autorizados.
4. **Intercambio de claves:** Utilizando criptografía asimétrica (RSA, ECDHE), las partes establecen una clave de sesión simétrica compartida sin transmitirla por la red.

5. **Comunicación cifrada:** Todos los datos subsecuentes se cifran con algoritmos simétricos eficientes (AES-GCM, ChaCha20-Poly1305) usando las claves de sesión establecidas.

Beneficios en IoT:

- **Confidencialidad:** Datos de sensores, comandos de actuadores, y telemetría permanecen privados incluso en redes no confiables.
- **Integridad:** Cualquier modificación de datos en tránsito es detectada inmediatamente.
- **Autenticación robusta:** Dispositivos y servidores verifican mutuamente sus identidades antes de intercambiar información sensible.

Consideraciones para dispositivos constrained:

Para dispositivos IoT con recursos limitados, existen variantes optimizadas:

- **DTLS (Datagram TLS):** Adaptación de TLS para UDP, útil en comunicaciones IoT donde la latencia es crítica.
- **TLS 1.3:** Versión más reciente con handshake simplificado (1-RTT o 0-RTT) que reduce overhead.
- **Compact TLS (cTLS):** Propuesta para reducir tamaño de mensajes de handshake.
- **Perfiles optimizados:** RFC 7925 define perfiles TLS/DTLS específicos para dispositivos IoT constrained.

Modos AEAD y Forward Secrecy

AEAD (Authenticated Encryption with Associated Data):

Los modos AEAD representan un avance significativo en criptografía simétrica al combinar cifrado y autenticación en una única operación primitiva. TLS 1.3 eliminó los modos de cifrado no-AEAD, mandatado exclusivamente cipher suites AEAD.

Características de AEAD:

- **Cifrado autenticado:** Proporciona simultáneamente confidencialidad (cifrado) e integridad/autenticidad (MAC) de los datos.
- **Associated Data:** Permite autenticar datos adicionales sin cifrarlos (como headers de protocolo), asegurando que el contexto completo del mensaje no ha sido alterado.
- **Protección contra manipulación:** Cualquier modificación del ciphertext, del nonce, o de los datos asociados resulta en fallo de verificación, previniendo ataques de bit-flipping, padding oracle, y otros ataques de manipulación.

Algoritmos AEAD comunes en TLS 1.3:

1. **AES-GCM (Galois/Counter Mode):**
    - AES-128-GCM y AES-256-GCM
    - Alto rendimiento con aceleración hardware (instrucciones AES-NI)

    - Estándar industrial ampliamente adoptado
2. **ChaCha20-Poly1305:**
    - Alternativa eficiente en software
    - Ideal para dispositivos sin aceleración hardware AES
    - Especialmente popular en dispositivos móviles y embedded
3. **AES-CCM (Counter with CBC-MAC):**
    - Usado en DTLS para IoT constrained
    - Menor overhead de RAM que GCM

Ventajas sobre modos no-AEAD:

Los modos legacy como CBC con MAC separado eran vulnerables a ataques de timing (Lucky13, BEAST, POODLE). AEAD elimina estas vulnerabilidades al integrar cifrado y autenticación de forma inseparable.

Forward Secrecy (Perfect Forward Secrecy - PFS):

Forward Secrecy garantiza que la exposición de claves privadas a largo plazo no comprometa sesiones pasadas. Esto se logra mediante el uso de claves de sesión efímeras que no derivan directamente de las claves privadas del certificado.

Mecanismos de Forward Secrecy:

1. **Ephemeral Diffie-Hellman (DHE):**
    - Genera nuevas claves Diffie-Hellman para cada sesión
    - Las claves de sesión se derivan del intercambio DHE
    - Incluso si la clave privada del servidor es comprometida posteriormente, las grabaciones de tráfico pasado no pueden descifrarse
2. **Elliptic Curve Diffie-Hellman Ephemeral (ECDHE):**
    - Variante usando curvas elípticas (más eficiente)
    - TLS 1.3 usa exclusivamente ECDHE, eliminando opciones sin PFS
    - Curvas recomendadas: X25519, P-256, P-384

TLS 1.3 y mejoras de seguridad:

- **Solo cipher suites con PFS:** Elimina RSA key exchange estático y otros métodos sin forward secrecy.
- **Handshake cifrado:** A partir del segundo mensaje, el handshake mismo está cifrado, protegiendo metadatos de configuración.
- **Eliminación de primitivas débiles:** Descarta SHA-1, MD5, RC4, DES, 3DES, y compresión TLS.
- **0-RTT con cautela:** Permite datos en el primer paquete (resumption) pero con advertencias de seguridad sobre replay attacks.

Beneficios para IoT:

En contextos IoT donde dispositivos pueden ser físicamente capturados años después de su despliegue:

- Las comunicaciones históricas permanecen protegidas incluso si se extraen las claves del dispositivo
- Los logs de tráfico capturado no pueden ser descifrados retroactivamente
- Se reduce el impacto de incidentes de seguridad a la ventana temporal de compromiso activo

Prevención de ataques con TLS

Protección contra Sniffing (Eavesdropping):

El sniffing es la captura pasiva de tráfico de red para extraer información sensible. Sin TLS, protocolos como HTTP, MQTT, CoAP transmiten datos en texto plano.

Mitigación mediante TLS:

- **Cifrado end-to-end:** Todo el payload está cifrado con algoritmos fuertes (AES-256, ChaCha20). Un atacante capturando paquetes solo ve datos aparentemente aleatorios.
- **Cifrado de metadatos de aplicación:** Headers HTTP, topics MQTT, URIs CoAP quedan protegidos dentro del túnel TLS.
- **Protección incluso en WiFi abierto:** TLS protege comunicaciones incluso cuando la red subyacente (WiFi pública) no está cifrada o usa cifrado débil (WEP/WPA comprometidos).

Limitación: TLS no oculta completamente los metadatos de tráfico (tamaños de paquetes, timing, endpoints IP). Para privacidad extrema, se requieren capas adicionales como VPN o Tor, pero esto excede los requisitos típicos de IoT.

Protección contra Man-in-the-Middle (MitM):

Los ataques MitM ocurren cuando un atacante se interpone entre cliente y servidor, interceptando, leyendo y potencialmente modificando comunicaciones.

Vectores de ataque MitM:

1. **ARP Spoofing:** Atacante redirige tráfico local haciéndose pasar por el gateway.
2. **DNS Spoofing:** Respuestas DNS falsas redirigen a servidores maliciosos.
3. **Rogue Access Points:** WiFi falsos que imitan redes legítimas.
4. **BGP Hijacking:** Redirección de tráfico a nivel de infraestructura internet.

Defensas TLS contra MitM:

1. **Autenticación mediante certificados:**
    - El cliente verifica que el certificado del servidor:
        - Está firmado por una CA confiable
        - Coincide con el hostname esperado
        - No ha expirado ni sido revocado
    - Esto previene que un atacante se haga pasar por el servidor legítimo

2. **Mutual TLS (mTLS):**
    - En IoT, el servidor también verifica el certificado del dispositivo
    - Previene dispositivos maliciosos imitando dispositivos legítimos
    - Especialmente crítico en contextos industriales y médicos
3. **Certificate Pinning:**
    - Dispositivos IoT pueden tener "pinned" certificados o claves públicas específicas
    - Rechaza conexiones incluso con certificados válidos si no coinciden con el pin
    - Protege contra compromisos de CA
4. **Verificación de cadena de confianza:**
    - Validación completa desde el certificado del servidor hasta la CA raíz
    - Detecta certificados auto-firmados o falsificados

Ejemplo de ataque prevenido:

Sin TLS: Atacante intercepta comando "ABRIR_PUERTA" a cerradura inteligente → lo ejecuta o modifica.

Con TLS: Atacante ve tráfico cifrado incomprensible → no puede leer el comando. Si intenta modificar bytes cifrados → verificación AEAD falla → conexión se cierra → ataque detectado.

Protección contra otros ataques:

- **Replay Attacks:** TLS incluye números de secuencia y timestamps. Retransmitir paquetes antiguos es detectado.
- **Downgrade Attacks:** TLS 1.3 elimina mecanismos de negociación inseguros. Intentos de forzar versiones antiguas son detectados mediante Signed Certificate Timestamps.
- **Truncation Attacks:** Mensajes de cierre están autenticados. Atacante no puede terminar prematuramente sesión sin detección.

Buenas prácticas de implementación:

1. **Usar TLS 1.3 o mínimo TLS 1.2** con cipher suites fuertes
2. **Habilitar certificate validation estricta** - nunca deshabilitar verificación en producción
3. **Implementar certificate pinning** cuando sea viable
4. **Rotación regular de certificados** y claves
5. **Monitoreo de intentos de handshake fallidos** como indicador de ataques
6. **HSTS (HTTP Strict Transport Security)** para forzar HTTPS en servicios web IoT
7. **Actualización oportuna** ante vulnerabilidades descubiertas (Heartbleed, POODLE, etc.)

Limitaciones de TLS:

- No protege contra malware en endpoints comprometidos
- No previene ataques de denegación de servicio (DoS)
- Requiere gestión adecuada de PKI (infraestructura de clave pública)
- Overhead computacional puede ser significativo en dispositivos muy constrained

Referencias web:

- **IoT Security Wiki**: https://iotsecuritywiki.com/
- **Security Research Labs - IoT**: https://srlabs.de/
- **CloudFlare - TLS 1.3 Overview**: https://blog.cloudflare.com/rfc-8446-aka-tls-1-3/